ChatGPT

Production Master Spec for a German Multi-Site Container Storage SaaS

Executive summary

The right benchmark for this product is not generic rental software and not a basic invoicing tool. It is a self-storage operating system that combines public storefront, live availability, online move-in, recurring billing, delinquency handling, access control, reporting and staff workflows in one platform. In the current market, is the clearest reference for an online-first storage operating system; is the strongest DACH-facing localisation reference because its German materials explicitly mention SEPA, invoice billing, DATEV export, EU hosting and GDPR; and wider set the bar for mature access, collections and role controls; and show practical operator workflows; and is helpful for booking UX, but remains generic-rental-first rather than storage-native. [1]

The German market signals a specific user expectation pattern: secure and dry storage, easy online booking, drive-up convenience for container-style sites, 24/7 or broad access hours, digital access credentials, flexible terms, and support for both private and business customers. The German self-storage association reports 115 members, 275 sites and around 1 million m² of lettable space, while German discovery sites distinguish between indoor self-storage, outdoor container storage and pickup-and-return services. Operator sites such as , , , and consistently surface the same promises: online booking, digital access, security, flexibility and private/business use. [2]

The legal and payment baseline is non-negotiable in the German market. Since 1 January 2025, domestic B2B invoicing in Germany generally requires structured electronic invoices; there are no exceptions to the ability to **receive** e-invoices; the structured part of an e-invoice must be retained for eight years; and a simple PDF is no longer an e-invoice under the new definition. SEPA Core and SEPA B2B direct debit have materially different refund and submission rules, so the billing engine must model mandate type, mandate evidence, pre-notification, pending settlement, retries and recovery explicitly. GDPR requires controller-processor contracts, purpose limitation, storage limitation, security measures, and deletion or anonymisation when data is no longer necessary unless a legal retention duty or legal claim basis applies. [3]

The product thesis is therefore this: build a German, multi-site, container/self-storage operating system that is online-first for tenants, low-friction for operators, portfolio-native for owners, and explicitly designed for SEPA, e-invoicing, DATEV-oriented accounting export, access automation, and container-specific inspections and yard mapping. The strongest build strategy is a modular monolith on a React [4] + Node.js [5] + TypeScript [6] stack with a single versioned master spec file that you update in place. This report includes that initial spec as **Master Spec v0.1.0**.

Market context and regulatory constraints

The German market is no longer a fringe niche. The Verband deutscher Self Storage Unternehmen e.V. [7] states that its network covers 115 members, 275 sites and about 1 million square metres of rentable space. Its public quality positioning emphasises controlled access, video surveillance, dryness, cleanliness and flexibility for both private and business customers. That matters because it means your

platform must support both consumer and company workflows from the start and must treat security and access as core product surfaces, not later add-ons. [8]

German buyer expectations are especially clear in container-led storage. advertises immediate online rental, 24/7 access, digital access control and simple cancellation; emphasises 24/7 access, digital access and online booking; focuses on direct vehicle access, fenced and access-controlled sites, insurance options and online booking; and adds insurance, packaging, transport help, package acceptance and private/business positioning. In practice, that means the buyer journey is not just “reserve a unit” but “solve a space problem immediately with confidence”. [9]

The category structure also matters. explicitly distinguishes self-storage boxes, self-storage containers, pickup-and-return storage, open storage and archive storage. For your product, that is important because it suggests the platform should model **site type** and **unit logistics model** from the beginning, even if v0.1.0 only fully supports fixed-site container and self-storage operations. It avoids painting yourself into a corner if later you add collection/delivery or document/archive workflows. [10]

German invoicing rules materially shape the product. The German Federal Ministry of Finance says that since 1 January 2025, domestic B2B transactions generally require structured e-invoices; a plain PDF is no longer an e-invoice; there are no exceptions to the obligation to **receive** e-invoices; and the structured part of the invoice must be retained unaltered for eight years. The same FAQ also says that an e-mail inbox is sufficient to receive e-invoices, and that formats fulfilling EN 16931 are the relevant structured standard. The European Commission’s eInvoicing materials describe EN 16931 as the harmonised European e-invoicing standard and explain that it exists precisely so invoice content can be processed electronically and interoperably. [11]

SEPA is equally product-shaping. The Deutsche Bundesbank states that SEPA direct debit in Germany has both Core and B2B variants; a Core debit can generally be refunded by the payer within eight weeks, while authorised B2B direct debits do not provide the same refund right; unauthorised debits can still be contested for up to 13 months; and the creditor identifier together with the mandate reference uniquely identifies the mandate. The European Payments Council adds that B2B is for business payers only and that the payer is not entitled to a refund for an authorised B2B transaction. Those are not back-office trivia. They determine billing states, customer messaging, access lockout timing and reconciliation logic. [12]

On privacy and security, the European Data Protection Board states that processors have direct GDPR obligations, that they must work under controller-processor contracts, and that processors must assist controllers with security, breach notification, rights handling and deletion/return of personal data. The EDPB and Commission materials also restate the storage-limitation principle: personal data should be kept only as long as necessary and then deleted or anonymised unless a legal retention obligation applies. This means the spec must clearly separate accounting retention from operational convenience, and it must include deletion, anonymisation and legal-hold mechanisms rather than generic “keep everything forever” behaviour. [13]

For signatures, the legal baseline comes from eIDAS. Article 25 says an electronic signature cannot be denied legal effect solely because it is electronic, while a qualified electronic signature has the equivalent legal effect of a handwritten signature. The Commission’s explanatory materials also make clear that qualified signatures are the safest cross-border option where handwritten-equivalence matters. For ordinary storage rental agreements, that supports a layered design: default to an evidence-rich electronic signature workflow and make qualified signatures an optional stricter mode rather than the universal baseline. [14]

Competitor feature mapping and product lessons

The current software landscape is uneven. Some platforms are excellent at storefront, move-in and automation; others are strongest in multi-site operations or access control; and some are generic rental tools that stop short of storage-native needs such as delinquency lockouts, unit maps and recurring tenancy billing. The table below compresses the most relevant signals from official product and help documentation.

Platform	Strongest confirmed capabilities	Product lesson for this SaaS
	Website and online checkout, built-in CRM, facility maps, automated payments, contracts and ID checks, access integrations, public API/webhooks, dynamic pricing, scheduled rent increases, custom roles from 80+ permissions, automated overlock on arrears	Best benchmark for an online-first, portfolio-aware storage operating system. Copy the integrated model, not just the feature list.
	Contactless online bookings, customer portal, e-sign, multi-site, interactive site map, automatic access denial on overdue accounts, tasks/reminders, full audit trail with notes/images, custom fields, open API/webhooks, German-language positioning around SEPA, invoice billing, DATEV export, EU hosting and GDPR	Closest current fit for a German operator. Strong reference for localisation, extensibility and auditability.
/	Highly configurable workflows, advanced reporting, browser/mobile operations, online move-in, e-sign, tenant portal credentials, access control, automatic lockouts, collections tooling, owner/minimal-access roles, assigned-facility scoping	Strong enterprise reference for mature role design, access workflows and collections.
	Single login for multi-location, integrated website/access/payments stack, conditional intake logic, move-in wizard, gate integrations, delinquency automation, invoice exports, standard staff/accountant roles, QuickBooks integration	Good reference for practical operator tasks, but less clearly evidenced as an open, modular platform.
	Contactless move-in, online payments, autogate integration, electronic signatures, mobile app, multi-site switching, 250+ reports, custom reporting	Good reference for operational depth and reporting breadth.
	Rental website builder, online bookings and payments, real-time availability, quotes, contracts, invoices, API keys, permission-scoped integrations	Excellent booking UX reference, but not a sufficient storage-native operating model by itself.

The mapping above is drawn from official product and help documentation. [15]

A more useful design table is the **priority ladder**: what to emulate immediately, what to adapt for the container niche, and what to defer.

Priority	Capability	Why it matters in this niche	Best references
Must build now	Live availability, online reservation, online move-in, e-sign, tenant portal, recurring billing, delinquency workflows, access automation, multi-site dashboards	These are the minimum features that modern operators and end customers already expect	, ,
Must adapt for the niche	Yard map, drive-up unit logic, container inspections with photos, waterproofing/ condition tracking, gate-plus-unit credential rules, business customer accounts with invoice export	These are where container sites differ materially from generic indoor-only storage or equipment rental	, , ,
Should build early	Public API, signed webhooks, fine-grained RBAC, accounting exports, rate management, tasking, incident workflows	These make the system production-grade, maintainable and partner-integrable	, ,
Defer	Managed marketing services, insurance marketplace, public marketplace distribution, lien/auction tools, full delivery logistics	Valuable later, but not core to a two-site German container operator's first production system	, ,

The most important analytical conclusion is this: **Storeganise is the nearest localisation reference, Stora is the strongest modern systems reference, and Storable is the mature enterprise reference**. That combination suggests your product should not try to out-feature every incumbent on day one. It should instead combine Stora's integrated operating model, Storeganise's German-market readiness, and Storable's access/collections/RBAC maturity, then add container-specific inspections, drive-up flow design and German accounting reality. That is a sharper wedge than merely "another self-storage SaaS". [16]

Product thesis, naming, permissions and technical direction

The product should use a simple, stable vocabulary. **Owner** is the business or portfolio administrator. **Operator** is staff running a site day to day. **Customer** is the umbrella commercial record. **Tenant** is a customer with at least one active rental agreement. **Organisation account** is a business customer with one or more contacts. This vocabulary avoids the ambiguity of "client", which can mean prospect, payer or active renter. It also aligns better with how mature storage software separates ownership, staff and active renters in practice. [17]

The UI split should follow those roles. The Owner experience should be portfolio-first: occupancy, revenue, overdue exposure, move-ins, move-outs, lead volume, pricing controls, site comparison, user administration, export jobs, integration health and audit. The Operator experience should be queue-first: leads, reservations, agreements pending signature, invoices requiring action, lockouts, access incidents, tasks, inspections, transfers and move-outs. The Tenant experience should be self-service-first: browse, reserve, sign, pay, manage mandate or card, view invoices, view access details, request move-out and contact support. Existing platforms have already trained users to expect those separations. [18]

The permissions model should be role-template plus per-site scoping. Mature systems already support site assignment, owner/minimal-access presets or highly granular permission sets. For your product, the surface area is large enough that “admin or not” will become a security and support problem very quickly. Even if Owner, Operator and Tenant remain the primary visible user types, the internal permission framework should already support future subroles such as Finance, Site Manager, Support and Read Only. [19]

The recommended build is a modular monolith using React [4] on the front end and Node.js [5] with TypeScript [6] on the back end, backed by PostgreSQL [20] as the system of record, an S3-compatible object store in an EU region for documents and photos, and a queue layer for billing, notifications, exports and webhook delivery. Identity should be OIDC-based with strict MFA for Owner and Operator users. Payments should be abstracted behind a gateway adapter because the storage domain needs cards, wallets, invoice/manual methods and SEPA direct debit, while true SEPA B2B support may differ by provider. This is a design recommendation grounded in the product’s required reliability and legal scope rather than a claim about any single vendor.

For payments, the safest recommendation is to support a primary PSP abstraction that can accommodate both cards and SEPA direct debit, and to avoid hard-coding the product around a single processor’s limitations. explicitly documents SEPA Direct Debit Core support, mandate collection and customer authorisation; documents recurring SEPA direct debit for European merchants; and makes clear that it currently offers SEPA Core rather than B2B. That is exactly why mandate type belongs in your domain model. [21]

For German accounting, the system should offer two export paths from the start: a durable file export path and an optional later service integration path. documents both the Buchungsdatenservice and Rechnungsdatenservice 1.0, and it also notes that the underlying DATEV-Format can be used as a direct file interface. That makes the right product strategy obvious: ship reliable DATEV-oriented exports in MVP, then add service integration only when the operator and accountant want deeper automation. [22]

Master Spec v0.1.0

File header

File path: `/docs/master-spec.md`
Canonical status: single source of truth
Version: `0.1.0`
Last updated: `2026-05-09 Europe/Berlin`
Product name: `Container OS` working title
Scope: German-first, multi-site container/self-storage SaaS for a two-site operator
Visible user types: `Owner`, `Operator`, `Tenant`
Internal commercial nouns: `Lead`, `Customer`, `Tenant`, `Organisation account`, `Contact`, `Reservation`, `Agreement`, `Invoice`, `Mandate`, `Access credential`, `Incident`
Rule: this file is always updated in place; do not fork the spec into multiple competing files. Every change must update version and changelog in this same file.

Changelog template

Use semantic versioning as follows:

- `0.1.x` research baseline, wording, scope clarifications
- `0.2.x` workflow and data-model refinements from operator interviews
- `0.3.x` API and UI contract stabilisation
- `0.4.x` integration and infra hardening
- `1.0.0` build-ready specification approved for implementation

Template:

```
## Changelog

### [0.1.0] - 2026-05-09
Added
- Initial production master spec
- Germany-first legal/payment baseline
- Three-role core model
- Module definitions and API surface

Changed
- N/A

Removed
- N/A

Open questions
- See open questions section
```

Assumptions

- The initial operator runs **two fixed sites**.
- The business rents storage on a recurring basis, mainly monthly.
- Both private individuals and business customers must be supported.
- Site access is at least partly digitised, especially gate access.
- The first production scope is fixed-site storage, not full pickup-and-return logistics.
- Containers and some classic self-storage units may coexist in the same portfolio.
- German e-invoicing, SEPA handling and DATEV-oriented exports are in scope from the first production design. [23]

Open questions

- Does the operator want pure month-to-month rentals, fixed terms, or both?
- Are deposits mandatory for all units or only selected sites/unit types?
- Is access controlled only at gate level, or also at row/zone/unit level?
- Which access vendors are already installed at the two sites?
- Are business customers expected to have multiple authorised site users under one account?
- Is invoice dispatch required in both German and English?
- Does the operator require true SEPA B2B direct debit, or is SEPA Core sufficient initially?

- Will the accountant expect file export only, or direct DATEV service integration later?
- Is ID verification mandatory for all move-ins or only for remote/contactless move-ins?
- Are insurance or protection-plan upsells part of the commercial model in phase one?
- Does the operator need vehicle storage, parking or trailer storage in the same system?
- Does the operator need package acceptance, transport help scheduling or merchandise sales like locks and boxes?

Global architectural conventions

API style

- Public endpoints: `/api/public/v1/*`
- Operator endpoints: `/api/operator/v1/*`
- Tenant endpoints: `/api/tenant/v1/*`
- Integration endpoints: `/api/system/v1/*`
- JSON over HTTPS
- UTC timestamps in storage; Europe/Berlin rendered in UI
- Monetary values stored as integer minor units
- All POST endpoints accept `Idempotency-Key`
- Webhooks signed with HMAC-SHA256
- All mutable entity changes emit audit events

Authentication

- Owner and Operator: OIDC login, MFA required
- Tenant: email/password or passwordless magic link; MFA optional for portal accounts
- API access: scoped bearer tokens for first-party and partner clients
- Session types: browser session for apps, bearer token for programmatic access

Multi-site model

- `workspace` = operator business
- `site` = physical location
- `zone` / `lane` / `block` = optional spatial subdivision
- `unit` = rentable storage object
- All operator permissions are site-scoped or workspace-scoped

Data handling

- Soft delete by default on commercial records
- Hard delete only where legally safe and no retention or legal-hold requirement exists
- Legal hold overrides deletion/anonymisation
- Encryption at rest for database, object storage and backups
- Virus scanning on uploaded files
- Immutable audit stream for admin actions

Permissions matrix

Action	Owner	Operator	Tenant
View portfolio dashboard	Yes	Site scope only	No

Action	Owner	Operator	Tenant
Create/edit sites	Yes	No	No
Create/edit units	Yes	Yes (site scope)	No
Set prices/promotions	Yes	Limited by policy	No
View and manage leads	Yes	Yes	Own enquiries only
Create manual reservations	Yes	Yes	Own only
Activate agreements	Yes	Yes	No
Sign agreements	No	No	Own only
Issue invoices/credits	Yes	Yes if permitted	No
Export accounting data	Yes	Finance-enabled only	No
Manage access integrations	Yes	Yes if permitted	No
Issue/revoke gate credentials	Yes	Yes if permitted	Own credentials view only
Complete inspections/tasks	Yes	Yes	Request only
Manage documents/templates	Yes	Limited by policy	Upload/download own
Manage users and roles	Yes	No by default	No
View audit log	Yes	Read-only limited	Own account activity only
View own invoices and receipts	No	No	Yes
Update own payment method	No	Assisted only	Yes
Request move-out	No	No	Yes

This matrix reflects the visible three-role model and leaves room for internal subroles like Finance or Site Manager later. The need for facility assignment and configurable role depth is already proven in current storage software. [19]

Global service levels and non-functional requirements

- Public booking availability SLO: **99.95%** monthly
- Operator and tenant app availability SLO: **99.9%** monthly
- RPO: **15 minutes**
- RTO: **4 hours**
- Public availability lookup p95: **<300 ms**
- Operator read APIs p95: **<500 ms**
- Booking completion p95 excluding PSP latency: **<1.5 s**
- Webhook dispatch: first attempt within **60 seconds**
- Queue backlog alerting at **5 minutes**
- Audit event write success: **99.99%**
- Zero silent data loss tolerance for billing, agreements and access state

Key flow diagrams

Booking to move-in

```
flowchart LR
  A[Public site page] --> B[Check live availability]
  B --> C[Create reservation or checkout session]
  C --> D[Collect customer details]
  D --> E[Collect payment method or SEPA mandate]
  E --> F[ID verification if policy requires]
  F --> G[Render agreement for e-sign]
  G --> H[Activate agreement]
  H --> I[Allocate unit]
  I --> J[Issue access credential]
  J --> K[Send move-in instructions]
  K --> L[Unit status becomes occupied]
```

Billing failure to lockout to recovery

```
flowchart LR
  A[Invoice created] --> B[Automatic charge attempt]
  B -->|Success| C[Invoice paid]
  B -->|Pending direct debit| D[Pending settlement]
  B -->|Failure| E[Retry policy starts]
  D -->|Clears| C
  D -->|Fails| E
  E --> F[Reminder notices]
  F --> G[Invoice overdue threshold reached]
  G --> H[Lockout or access suspension]
  H --> I[Customer updates payment method or pays]
  I --> J[Reconciliation confirms payment]
  J --> K[Restore access and close delinquency state]
```

Access event to incident

```
flowchart LR
  A[Gate or unit access event] --> B{Normal?}
  B -->|Yes| C[Store access log]
  B -->|No| D[Create incident]
  D --> E[Assign operator task]
  E --> F[Review logs, photos, notes]
  F --> G[Take action: unlock, suspend, escalate, resolve]
  G --> H[Incident closed with audit trail]
```

Representative domain schemas

```
{
  "Site": {
    "id": "site_01",
    "name": "Passau Hafen",
    "slug": "passau-hafen",
    "currency": "EUR",
    "timezone": "Europe/Berlin",
    "status": "active"
  },
  "Unit": {
    "id": "unit_01",
    "siteId": "site_01",
    "unitCode": "A-101",
    "kind": "container",
    "status": "available",
    "sizeSqm": 14.0,
    "driveUp": true
  },
  "Reservation": {
    "id": "res_01",
    "siteId": "site_01",
    "unitId": "unit_01",
    "status": "confirmed",
    "startDate": "2026-06-01"
  },
  "Agreement": {
    "id": "agr_01",
    "reservationId": "res_01",
    "tenantId": "ten_01",
    "status": "active",
    "billingMode": "monthly"
  },
  "Invoice": {
    "id": "inv_01",
    "agreementId": "agr_01",
    "status": "pending",
    "totalMinor": 14900,
    "currency": "EUR"
  },
  "AccessCredential": {
    "id": "acc_01",
    "agreementId": "agr_01",
    "credentialType": "pin",
    "status": "active"
  }
}
```

Module specifications

Site and inventory

Purpose
Manage all physical locations, maps, zones and rentable units, including container-specific metadata and real-time occupancy state.

Data model
Entities and key fields:

- `Site` : `id` , `name` , `slug` , `timezone` , `currency` , `address` , `accessHours` , `status`
- `Zone` : `id` , `siteId` , `name` , `mapPolygon` , `vehicleAccess`
- `UnitType` : `id` , `siteId` , `name` , `kind` , `sizeSqm` , `sizeCbm` , `doorType` , `features[]`
- `Unit` : `id` , `siteId` , `zoneId` , `unitCode` , `unitTypeId` , `kind` , `status` , `driveUp` , `position` , `photoUrl` , `conditionState` , `lastInspectionId`
- `Amenity` : `id` , `siteId` , `name` , `displayOrder`
- `InventoryEvent` : `id` , `unitId` , `oldStatus` , `newStatus` , `reason` , `actorId` , `createdAt`

API surface

- `GET /api/public/v1/sites`
- `GET /api/public/v1/sites/{slug}/unit-types`
- `GET /api/public/v1/sites/{slug}/availability?startDate=...`
- `POST /api/operator/v1/sites`
- `PATCH /api/operator/v1/sites/{siteId}`
- `POST /api/operator/v1/sites/{siteId}/units/import`
- `GET /api/operator/v1/units?siteId=...`
- `PATCH /api/operator/v1/units/{unitId}`
- `POST /api/operator/v1/units/{unitId}/status-transition`

Auth:

- Public browse: none
- Owner: workspace scope
- Operator: site scope

Sample request/response:

```
{
  "request": {
    "siteId": "site_01",
    "unitCode": "A-101",
    "unitTypeId": "ut_container_20ft",
    "kind": "container",
    "driveUp": true,
    "position": {"row": "A", "index": 101},
    "status": "available"
  },
```

```
  "response": {
    "id": "unit_01",
    "siteId": "site_01",
    "unitCode": "A-101",
    "kind": "container",
    "status": "available",
    "auditVersion": 1
  }
}
```

UI flows and wireframes

- Owner: portfolio map card at top, site selector, occupancy heatmap, “bulk import/edit units” panel on the right, exceptions list at bottom.
- Operator: unit list left, map/yard view centre, selected unit detail drawer right, quick actions for reserve, move-in, transfer, maintenance.
- Tenant: public site page with site photos, size cards, availability badges, price from, access hours, FAQ.

Error cases

- Duplicate `unitCode` within the same site
- Attempt to move occupied unit to deleted/out-of-service state without transfer/move-out
- Bulk import row validation errors
- Map coordinate mismatch or invalid spatial reference

Business rules

- `unitCode` must be unique per site.
- Status changes are constrained: `occupied` cannot jump directly to `available` without move-out.
- A unit with historical agreements cannot be hard-deleted.
- Container units require container-specific fields: `driveUp`, `weatherSealState`, `doorType`, `groundAccessType`.

Retention and GDPR

- Inventory records are operational and historical, not directly personal.
- Links from unit history to tenants must respect anonymisation rules when tenant PII is later minimised.
- Site and unit audit history is retained for operational accountability. [24]

Test cases

- Import 500 units with mixed statuses
- Reserve last available unit while another operator edits metadata
- Transfer a unit from maintenance back to availability
- Render live availability correctly on public site page

Monitoring and SLAs

- Site and availability read p95 under 300 ms

- Inventory change propagation to public availability under 5 seconds
- Bulk import success/failure metrics and row-level traceability

Non-functional requirements

- Support at least 20,000 units per workspace
- Optimistic locking on unit edits
- Bulk actions with rollback report
- Spatial map data versioning

Pricing and revenue

Purpose
Control rent, promotions, fees, rent increases, discounts and tax/accounting mapping.

Data model

- `PriceBook`: `id`, `siteId`, `name`, `status`, `effectiveFrom`
- `RateRule`: `id`, `priceBookId`, `unitTypeId`, `amountMinor`, `billingCycle`, `conditions`
- `Promotion`: `id`, `code`, `discountType`, `value`, `stackingPolicy`, `validFrom`, `validTo`
- `FeeSchedule`: `id`, `siteId`, `depositMinor`, `lateFeePolicy`, `adminFeeMinor`
- `RentIncreasePolicy`: `id`, `siteId`, `frequency`, `noticeDays`, `formula`
- `TaxProfile`: `id`, `siteId`, `taxCode`, `vatRate`

API surface

- `GET /api/public/v1/quotes`
- `POST /api/operator/v1/price-books`
- `POST /api/operator/v1/price-books/{id}/publish`
- `POST /api/operator/v1/pricing/simulate`
- `PATCH /api/operator/v1/promotions/{promoId}`

Auth:

- Public quote: none
- Owner/Operator: pricing permissions

Sample request/response:

```
{
  "request": {
    "siteId": "site_01",
    "unitTypeId": "ut_container_20ft",
    "startDate": "2026-06-01",
    "customerType": "business",
    "promoCode": "SPRING15"
  },
  "response": {
    "currency": "EUR",
```

```
    "rentMinor": 14900,
    "depositMinor": 14900,
    "feesMinor": 0,
    "discountMinor": 2235,
    "totalDueTodayMinor": 27565
  }
}
```

UI flows and wireframes

- Owner: pricing matrix by site/unit type with “draft vs published” toggle, promotion panel, future rent increases calendar.
- Operator: manual quote simulator with override note field.
- Tenant: simple quote card in checkout showing rent, deposit, tax and first payment due.

Error cases

- Overlapping active rate rules
- Promotion expired or not applicable
- Negative final price after stacked discounts
- Missing VAT/accounting mapping

Business rules

- Only one published rate rule per site/unit type/date combination.
- Promotion stacking policy must be explicit.
- Rent increase notices are generated before effective date.
- Historical invoices always keep original line calculation snapshots.

Retention and GDPR

- Pricing data is commercial configuration, not personal data.
- Quote events tied to identifiable leads follow lead retention policy.

Test cases

- Promotion stacking and non-stacking
- Mid-month proration
- Scheduled rent increase notice generation
- Business vs consumer VAT/accounting handling

Monitoring and SLAs

- Quote calculation p95 under 250 ms
- Publish job completes and invalidates caches within 1 minute

Non-functional requirements

- Deterministic price engine
- Fully reproducible historical calculations
- Snapshot pricing into reservation and invoice contexts

Sales and storefront

Purpose
Turn website traffic into reservations, move-ins and qualified leads.

Data model

- `LandingPageConfig`: `siteId`, `heroContent`, `faqBlocks`, `seoMeta`
- `CheckoutSession`: `id`, `siteId`, `unitTypeId`, `state`, `expiresAt`
- `QuoteRequest`: `id`, `siteId`, `contact`, `requirements`, `status`
- `StorefrontEvent`: `id`, `type`, `sessionId`, `siteId`, `metadata`

API surface

- `GET /api/public/v1/storefront/{siteSlug}`
- `GET /api/public/v1/storefront/{siteSlug}/availability`
- `POST /api/public/v1/checkout-sessions`
- `POST /api/public/v1/quote-requests`
- `POST /api/public/v1/storefront-events`

Auth:

- Public for browse and create session
- Operator/Owner for CMS controls only

Sample request/response:

```
{
  "request": {
    "siteSlug": "passau-hafen",
    "unitTypeId": "ut_container_20ft",
    "startDate": "2026-06-01"
  },
  "response": {
    "checkoutSessionId": "chk_01",
    "expiresAt": "2026-05-09T12:15:00Z",
    "availabilityState": "available"
  }
}
```

UI flows and wireframes

- Owner: SEO and conversion settings page, site content blocks, tracking integrations, abandonment metrics.
- Operator: manual shareable checkout link generator for phone-assisted sales.
- Tenant: choose site → choose size → price summary → customer details → payment/mandate → sign → confirmation.

Error cases

- Unit type sells out mid-checkout
- PSP or mandate setup fails
- Session expiry during long form fill
- Invalid browser state after refresh

Business rules

- Live availability is authoritative.
- Checkout sessions expire automatically.
- Abandoned sessions can create leads if consent exists.
- Public site pages must remain accessible even when a site is temporarily non-bookable.

Retention and GDPR

- Abandoned sessions retain only minimal data unless the user submits identifying information or consents to follow-up.
- Analytics and attribution must separate strictly necessary operational telemetry from marketing consented tracking. [25]

Test cases

- Search-engine-friendly landing pages
- Session expiry recovery
- Mobile checkout on low-bandwidth connection
- Conversion from quote request to operator-assisted reservation

Monitoring and SLAs

- Public storefront uptime 99.95%
- Checkout step failure funnel
- Session expiration and sold-out race metrics

Non-functional requirements

- Mobile-first UX
- Fast first contentful render
- Graceful degradation if tracking scripts fail
- Strong anti-bot and form abuse protection

CRM and leads

Purpose
Capture, deduplicate, track and convert leads into customers and then into tenants.

Data model

- `Lead`: `id`, `siteId`, `source`, `status`, `intent`, `moveInDate`, `notes`
- `Customer`: `id`, `type`, `personOrOrgData`, `marketingConsent`
- `Contact`: `id`, `customerId`, `role`, `email`, `phone`
- `Activity`: `id`, `subjectType`, `subjectId`, `channel`, `body`, `actorId`

- `ConversationThread`: `id`, `customerId`, `channel`

API surface

- `POST /api/public/v1/leads`
- `GET /api/operator/v1/leads`
- `PATCH /api/operator/v1/leads/{leadId}`
- `POST /api/operator/v1/leads/{leadId}/activities`
- `POST /api/operator/v1/customers/merge`

Auth:

- Public create
- Operator site scope
- Owner workspace scope

Sample request/response:

```
{
  "request": {
    "siteId": "site_01",
    "name": "Anna Weiss",
    "email": "anna@example.com",
    "intent": "business_storage",
    "moveInDate": "2026-06-01"
  },
  "response": {
    "leadId": "lead_01",
    "status": "new",
    "nextSuggestedAction": "contact_within_15m"
  }
}
```

UI flows and wireframes

- Owner: pipeline dashboard by site/source, conversion funnel, abandoned checkout recovery list.
- Operator: lead inbox with timeline centre, notes and communication drawer right, quick actions for reserve, quote, call back.
- Tenant: support/contact page showing previous portal requests and responses.

Error cases

- Duplicate lead creation from repeated checkout attempts
- Lead merged into wrong customer
- Missing consent flag for marketing follow-up
- Spam submissions

Business rules

- One customer can have multiple contacts and multiple agreements.
- Lead conversion creates or links a customer record.

- Marketing consent is separate from transactional messaging.
- Dedupe uses email, phone and fuzzy name/address heuristics.

Retention and GDPR

- Non-converted leads should be auto-reviewed and deleted or anonymised after a configured period unless consent or legitimate-interest basis remains.
- Activity timelines should retain only what is necessary. [26]

Test cases

- Merge duplicate business accounts
- Convert lead to reservation from operator screen
- Consent-aware email segmenting
- Timeline completeness across channels

Monitoring and SLAs

- Lead creation alerting within 1 minute
- Duplicate-match rate
- Lead response-time tracking

Non-functional requirements

- Strong deduplication
- Searchable timeline
- Safe PII redaction in internal notes where needed

Reservations

Purpose
Hold inventory temporarily, capture intent and prepare the path to agreement activation.

Data model

- `Reservation`: `id`, `siteId`, `unitId`, `unitTypeId`, `customerId`, `status`, `startDate`, `expiresAt`
- `ReservationHold`: `id`, `reservationId`, `lockToken`, `expiresAt`
- `ReservationCharge`: `id`, `reservationId`, `type`, `amountMinor`

API surface

- `POST /api/public/v1/reservations`
- `POST /api/public/v1/reservations/{id}/confirm`
- `POST /api/public/v1/reservations/{id}/cancel`
- `GET /api/tenant/v1/reservations`
- `POST /api/operator/v1/reservations`

Auth:

- Public create/confirm for self-service
- Tenant for own reservations

- Operator for manual reservations

Sample request/response:

```
{
  "request": {
    "siteId": "site_01",
    "unitTypeId": "ut_container_20ft",
    "startDate": "2026-06-01"
  },
  "response": {
    "reservationId": "res_01",
    "status": "pending_signature",
    "expiresAt": "2026-05-09T12:15:00Z"
  }
}
```

UI flows and wireframes

- Owner: today's reservation board with ageing and conversion metrics.
- Operator: reservation list with expiry countdowns and "convert to active move-in" buttons.
- Tenant: reservation confirmation page showing countdown, next steps and documents still required.

Error cases

- Hold expires before confirmation
- Payment succeeds after hold expiry
- Reservation references manually removed inventory
- Start date violates site policy

Business rules

- Temporary online holds expire automatically.
- Expired reservations release inventory immediately.
- Reservation may require payment method or deposit depending on policy.
- Future move-in window is site-configurable.

Retention and GDPR

- Expired or cancelled reservations without active tenancy should follow lead/customer retention rules.

Test cases

- Simultaneous last-unit reservations
- Deposit-required reservation
- Future-dated move-in
- Reservation conversion to agreement

Monitoring and SLAs

- Average hold time
- Expiry-driven release success
- Sold-out race-condition metric

Non-functional requirements

- Idempotent confirmation flow
- Strong locking around inventory
- Clear recovery path after interruptions

Rental agreements

Purpose
Generate, sign, activate, amend and terminate rental contracts.

Data model

- `AgreementTemplate`: `id`, `siteId`, `version`, `language`, `body`
- `Agreement`: `id`, `reservationId`, `tenantId`, `status`, `effectiveFrom`, `billingCycle`, `terminationRules`
- `AgreementAmendment`: `id`, `agreementId`, `type`, `effectiveFrom`
- `Signatory`: `id`, `agreementId`, `personId`, `status`
- `TerminationRequest`: `id`, `agreementId`, `requestedDate`, `status`

API surface

- `POST /api/operator/v1/agreements/draft`
- `POST /api/operator/v1/agreements/{id}/send-for-signature`
- `POST /api/operator/v1/agreements/{id}/activate`
- `POST /api/tenant/v1/agreements/{id}/sign`
- `POST /api/tenant/v1/agreements/{id}/move-out-request`

Auth:

- Operator/Owner for drafting and activation
- Tenant for signature and self-service requests

Sample request/response:

```
{
  "request": {
    "reservationId": "res_01",
    "templateVersion": "de-2026-05",
    "billingCycle": "monthly"
  },
  "response": {
    "agreementId": "agr_01",
    "status": "pending_signature",
    "documentId": "doc_01"
```

```
  }
}
```

UI flows and wireframes

- Owner: template library with versions, locale variants and mandatory-clause toggles.
- Operator: agreement checklist view with identity, payment and signature status.
- Tenant: portal document viewer with agreement summary, signature action, downloadable PDF and evidence receipt.

Error cases

- Template version archived during signing
- Required signatory missing
- Activation attempted before signature or mandatory payments
- Move-out request conflicts with overdue state or notice period

Business rules

- Agreement becomes active only when all required prerequisites are complete.
- Amendments are versioned and linked to the original agreement.
- Important commercial terms are snapshotted onto billing records.
- Move-out requests can be tenant-initiated but may require operator confirmation.

Retention and GDPR

- Signed agreements and evidence packs are retained for the operational and legal-claims window, and for longer where tied to accounting evidence or legal hold.
- Access must be tightly role-scoped. 27

Test cases

- Self-service signing
- Operator-assisted in-office signing
- Agreement amendment on upsell/change of unit
- Move-out notice with required lead time

Monitoring and SLAs

- Signature success rate
- Template rendering failures
- Activation latency after all prerequisites complete

Non-functional requirements

- Version-safe template rendering
- Full evidentiary audit trail
- Multi-language support

Billing and mandates

Purpose
Generate invoices, manage recurring rent, handle SEPA mandates, reminder workflows, delinquency and lockout state.

Data model

- `Invoice`: `id`, `agreementId`, `status`, `invoiceDate`, `dueDate`, `currency`, `totalMinor`
- `InvoiceLine`: `id`, `invoiceId`, `kind`, `description`, `amountMinor`, `taxCode`
- `Mandate`: `id`, `customerId`, `scheme`, `reference`, `creditorId`, `status`, `signedAt`
- `ReminderPolicy`: `id`, `siteId`, `steps[]`
- `DelinquencyPolicy`: `id`, `siteId`, `overdueDays`, `lockoutEnabled`, `lateFeeRules[]`
- `CreditNote`: `id`, `invoiceId`, `amountMinor`, `reason`

API surface

- `POST /api/operator/v1/invoice-runs`
- `GET /api/operator/v1/invoices/{invoiceId}`
- `POST /api/operator/v1/invoices/{invoiceId}/charge`
- `POST /api/operator/v1/invoices/{invoiceId}/send-reminder`
- `POST /api/tenant/v1/mandates`
- `GET /api/tenant/v1/billing`

Auth:

- Owner/Operator for invoice operations
- Tenant for own billing and mandate setup

Sample request/response:

```
{
  "request": {
    "customerId": "cust_01",
    "scheme": "sepa_core",
    "ibanLast4": "1234",
    "consentSource": "checkout"
  },
  "response": {
    "mandateId": "man_01",
    "reference": "MAN-2026-0001",
    "status": "pending"
  }
}
```

UI flows and wireframes

- Owner: delinquency policy editor, invoice ageing dashboard, direct debit settlement summary.

- Operator: invoice detail page with timeline, retry, waive fee, manual payment and suspend/restore access controls.
- Tenant: billing portal with invoices, receipts, payment methods, mandate status and outstanding balance.

Error cases

- Duplicate invoice period
- Mandate setup incomplete
- Direct debit remains pending beyond configured threshold
- Retry attempted after invoice already settled
- Credit note exceeds original balance

Business rules

- System must support `sepa_core`, `sepa_b2b`, `card`, `manual_invoice`, `cash`, `bank_transfer`.
- PSP adapter may restrict which schemes are available; domain model must not.
- Pending settlement state must be explicit for direct debits.
- Overdue status is policy-based, not inferred purely from due date.
- Access lockout may occur automatically when overdue threshold is reached.

Retention and GDPR

- Structured e-invoice payloads and related invoice evidence must be retained for eight years in unaltered form where required.
- Since 1 January 2025, domestic B2B invoicing needs structured e-invoices and businesses must be able to receive them.
- Mandate evidence, creditor ID and mandate reference must be stored to support SEPA operations and disputes. [28]

Test cases

- Monthly rent generation on fixed date
- Anniversary billing mode if later enabled
- Direct debit pending → paid
- Direct debit pending → failed → retry policy
- Overdue invoice triggers lockout
- Payment clears and restores access

Monitoring and SLAs

- Nightly invoice run completes before 02:00 local time
- Direct debit pending ageing alarm
- Failed payment recovery rate
- Lockout action dispatch under 60 seconds after delinquency transition

Non-functional requirements

- Idempotent invoice generation
- Immutable invoice snapshots
- Exact reconciliation references

- Clear audit of every manual adjustment

Payments and accounting exports

Purpose
Process payments, reconcile gateway events, maintain a finance-safe ledger and provide German export paths including DATEV-oriented output.

Data model

- `Payment`: `id`, `invoiceId`, `method`, `status`, `amountMinor`, `reference`
- `PaymentAttempt`: `id`, `paymentId`, `provider`, `status`, `providerRef`, `errorCode`
- `LedgerEntry`: `id`, `type`, `refType`, `refId`, `debitAccount`, `creditAccount`, `amountMinor`
- `ExportJob`: `id`, `kind`, `scope`, `status`, `downloadUrl`, `createdAt`
- `AccountingMapping`: `id`, `siteId`, `revenueAccount`, `taxCode`, `costCenter`

API surface

- `POST /api/tenant/v1/payments/intents`
- `POST /api/system/v1/payments/webhooks/{provider}`
- `GET /api/operator/v1/ledger`
- `POST /api/operator/v1/exports/datev`
- `GET /api/operator/v1/exports/{exportId}`

Auth:

- Tenant for own payment initiation
- System webhook auth via signature
- Operator/Owner for exports and reconciliation

Sample request/response:

```
{
  "request": {
    "kind": "datev_bookings",
    "siteIds": ["site_01", "site_02"],
    "from": "2026-05-01",
    "to": "2026-05-31"
  },
  "response": {
    "exportJobId": "exp_01",
    "status": "queued"
  }
}
```

UI flows and wireframes

- Owner: finance dashboard, export job history, payout mismatch alerts, account mapping settings.

- Operator: failed payment work queue and guided manual reconciliation.
- Tenant: payment confirmation page, receipt history, saved method manager.

Error cases

- PSP webhook replay
- Unmatched payment event
- Export generated with incomplete account mapping
- Currency mismatch
- Duplicate settlement import

Business rules

- All finance writes post through the ledger service.
- Export path one: DATEV-format-compatible file output.
- Export path two: future direct service integration into DATEV.
- Accounting mapping must be effective-dated and historically reproducible.

Retention and GDPR

- Bookings, payment references and exported accounting data follow accounting retention duties.
- DATEV's official services support both transferred structured data and file-based formats, so both must remain auditable. [29]

Test cases

- Partial payment and balance carry-forward
- Payment refund with credit note
- Provider webhook retries
- DATEV export with mixed taxes and deposits

Monitoring and SLAs

- Webhook verification failures
- Reconciliation mismatch count
- Export job duration and failure reason metrics

Non-functional requirements

- Immutable ledger
- Traceable provider references
- Safe replay handling
- Downloadable export artefacts with checksum

Access control and integrations

Purpose
Issue, revoke and audit physical access credentials and integrate with gate/door vendors.

Data model

- `AccessVendor`: `id`, `name`, `adapterType`, `siteIds[]`

- `AccessPoint`: `id`, `siteId`, `vendorId`, `kind`, `name`, `state`
- `AccessGroup`: `id`, `siteId`, `rules`
- `AccessCredential`: `id`, `agreementId`, `credentialType`, `externalRef`, `status`
- `AccessEvent`: `id`, `accessPointId`, `credentialId`, `eventType`, `result`, `occurredAt`
- `LockoutState`: `id`, `agreementId`, `reason`, `active`

API surface

- `POST /api/operator/v1/access/credentials`
- `PATCH /api/operator/v1/access/credentials/{id}`
- `GET /api/operator/v1/access/events`
- `POST /api/system/v1/access/vendors/{vendor}/webhooks`
- `POST /api/operator/v1/access/points/{id}/remote-open`

Auth:

- Operator/Owner with access permissions
- Vendor webhook signature authentication
- Tenant read-only for own credential instructions

Sample request/response:

```
{
  "request": {
    "agreementId": "agr_01",
    "credentialType": "pin",
    "siteId": "site_01"
  },
  "response": {
    "credentialId": "cred_01",
    "status": "active",
    "maskedCredential": "****42"
  }
}
```

UI flows and wireframes

- Owner: integration dashboard by site with online/offline vendor state and failed sync count.
- Operator: access console with remote open, credential reissue, event stream and incident shortcut.
- Tenant: portal page showing access instructions, hours and credential display according to policy.

Error cases

- Vendor API unavailable
- Credential issued but not acknowledged by vendor
- Lockout status diverges between platform and vendor
- Duplicate PIN or invalid access-group assignment

Business rules

- Access derives from agreement status, site hours and delinquency state.
- Lockout must be reversible automatically after payment confirmation.
- Credentials are never shown in full to staff unless policy allows.
- Vendor adapters run with reconciliation jobs to heal drift.

Retention and GDPR

- Access logs contain behavioural data and must have configurable retention; keep only as long as needed for security, support and legal claims.
- Operators need access to logs, but access must be limited and auditable. [24]

Test cases

- Issue credential on activation
- Suspend after overdue threshold
- Restore after successful payment
- Vendor outage fallback and retry
- Access denied creates incident

Monitoring and SLAs

- Credential issue latency under 30 seconds from activation
- Vendor sync error rate
- Drift reconciler backlog
- Access event ingestion lag

Non-functional requirements

- Vendor-agnostic adapter pattern
- Signed webhook ingestion
- Replay-safe event handling
- High-confidence audit trail

Operations, tasks and inspections

Purpose
Run daily site operations including tasks, inspections, incidents, maintenance and transfers.

Data model

- `Task`: `id`, `siteId`, `assigneeId`, `status`, `dueAt`, `subjectRef`
- `InspectionTemplate`: `id`, `siteId`, `kind`, `checklist[]`
- `InspectionRun`: `id`, `unitId`, `kind`, `result`, `photoIds[]`
- `Incident`: `id`, `siteId`, `severity`, `type`, `status`, `linkedAccessEventId`
- `MaintenanceOrder`: `id`, `unitId`, `status`, `vendorContact`
- `Transfer`: `id`, `fromUnitId`, `toUnitId`, `agreementId`, `effectiveDate`

API surface

- `POST /api/operator/v1/tasks`

- `PATCH /api/operator/v1/tasks/{taskId}`
- `POST /api/operator/v1/inspections`
- `POST /api/operator/v1/incidents`
- `POST /api/operator/v1/transfers`

Auth:

- Operator for site work
- Owner for oversight
- Tenant only for issue/move-out request creation

Sample request/response:

```
{
  "request": {
    "unitId": "unit_01",
    "kind": "move_in",
    "checklist": [
      {"code": "dry", "result": "pass"},
      {"code": "door_seal", "result": "pass"}
    ]
  },
  "response": {
    "inspectionId": "ins_01",
    "result": "pass"
  }
}
```

UI flows and wireframes

- Owner: operations board grouped by site, incident severity and ageing.
- Operator: daily job list on the left, selected task/inspection in centre, photo upload and notes drawer on the right.
- Tenant: request move-out or report problem from portal.

Error cases

- Transfer target is not available
- Mandatory inspection missing for container move-in
- Incident closed without resolution note
- Photo uploads fail mid-inspection

Business rules

- Container move-in and move-out inspections are mandatory by default.
- Failed move-out inspection can block or reduce deposit release subject to policy.
- Scheduled transfers preserve billing history and generate new access credentials if needed.
- Task completion is audited.

Retention and GDPR

- Inspection photos and notes may contain personal or sensitive incidental information; minimise capture and apply defined retention policies.
- Incident data tied to disputes may require legal-hold support. [30]

Test cases

- Move-in inspection with mandatory photo
- Transfer agreement to larger unit
- Access denial incident creation
- Overdue task summary generation

Monitoring and SLAs

- Open incident ageing
- Task backlog by site
- Inspection completion before move-in cutoff

Non-functional requirements

- Offline-tolerant upload queue for mobile use
- Photo checksum validation
- Strong subject linking to agreements/units/incidents

Documents, e-sign and ID verification

Purpose
Store business documents, render signatures, collect identity evidence where policy requires and maintain evidentiary packs.

Data model

- `Document`: `id`, `subjectType`, `subjectId`, `kind`, `storageKey`, `hash`
- `DocumentTemplate`: `id`, `name`, `locale`, `version`
- `SignatureEnvelope`: `id`, `documentId`, `provider`, `status`
- `EvidencePack`: `id`, `documentId`, `events[]`, `hash`
- `VerificationSession`: `id`, `customerId`, `provider`, `status`
- `VerificationResult`: `id`, `sessionId`, `outcome`, `riskFlags[]`

API surface

- `POST /api/operator/v1/documents/upload`
- `POST /api/operator/v1/signatures/send`
- `POST /api/tenant/v1/signatures/{envelopeId}/complete`
- `POST /api/tenant/v1/id-verification/sessions`
- `GET /api/tenant/v1/documents`

Auth:

- Operator/Owner for template and admin operations
- Tenant for own uploads and signatures

Sample request/response:

```
{
  "request": {
    "customerId": "cust_01",
    "documentKind": "id_front",
    "fileName": "id.jpg"
  },
  "response": {
    "documentId": "doc_01",
    "uploadUrl": "signed-upload-url",
    "status": "awaiting_upload"
  }
}
```

UI flows and wireframes

- Owner: template/version manager and policy settings for when ID verification is required.
- Operator: checklist panel showing missing documents/signature/verification items.
- Tenant: clear step-by-step upload and signature flow with status bars and downloadable completed documents.

Error cases

- Unsupported file type
- Virus scan failure
- Signature provider timeout
- Verification inconclusive
- Template variable missing

Business rules

- Signature mode can be `simple`, `advanced`, or `qualified`.
- Default move-in flow uses evidence-rich electronic signature unless higher assurance is requested.
- ID verification is policy-driven and optional by site/risk rule.
- Every signed document stores a hash and audit evidence pack.

Retention and GDPR

- eIDAS permits electronic signatures to have legal effect; qualified signatures are equivalent to handwritten signatures.
- Keep verification data only as long as necessary for fraud prevention, compliance and claims handling. [31]

Test cases

- Self-service document upload and signing
- Failed virus scan
- Re-sign after template correction
- Optional ID verification skip for in-person move-in

Monitoring and SLAs

- Upload errors by file type
- Signature completion rate
- Verification provider latency

Non-functional requirements

- EU-region object storage
- Virus scanning and hash verification
- Evidence-pack immutability
- Large-file resumable uploads

Reporting and analytics

Purpose
Provide live operational dashboards and financial reporting across sites.

Data model

- `DashboardPreset`: `id`, `role`, `widgets[]`
- `MetricSnapshot`: `id`, `siteId`, `metric`, `value`, `bucketAt`
- `ReportRun`: `id`, `kind`, `params`, `status`
- `CohortDefinition`: `id`, `kind`, `filters`

API surface

- `GET /api/operator/v1/dashboards/owner`
- `GET /api/operator/v1/dashboards/operator`
- `GET /api/operator/v1/reports/occupancy`
- `GET /api/operator/v1/reports/revenue`
- `POST /api/operator/v1/reports/custom-export`

Auth:

- Owner and Operator with report scopes
- Tenant limited to own account history only

Sample request/response:

```
{
  "request": {
    "report": "occupancy",
    "siteIds": ["site_01", "site_02"],
    "from": "2026-05-01",
    "to": "2026-05-31"
  },
  "response": {
    "rows": [
      {"siteId": "site_01", "occupancyPct": 87.5},
      {"siteId": "site_02", "occupancyPct": 92.1}
```

```
      ]
    }
  }
```

UI flows and wireframes

- Owner: portfolio KPI header, site comparison cards, trend charts, overdue risk and bookings by source.
- Operator: site dashboard with today’s move-ins/move-outs, overdue list and incident queue.
- Tenant: account summary with invoices, receipts, active rental and access history.

Error cases

- Stale aggregate data
- Missing site permission for requested report
- Large export timing out

Business rules

- Operational dashboard may read from cache, but core financial reports must reconcile to ledger source of truth.
- Time grouping uses Europe/Berlin in UI.
- Every report should expose its data freshness timestamp.

Retention and GDPR

- Aggregated analytics should be preferred over indefinite raw personal event retention.
- Raw identifiable event retention should be time-bound and reviewed. 26

Test cases

- Site-scoped operator dashboard
- Portfolio comparison for owner
- Large CSV export
- Revenue reconciliation between ledger and dashboard

Monitoring and SLAs

- Dashboard render p95 under 2 seconds for cached views
- Data freshness lag under 5 minutes for operational metrics
- Export queue time

Non-functional requirements

- Clear freshness metadata
- Exportable CSV/XLSX
- Drill-down from aggregate to operational records
- Stable metric definitions

API and webhooks

Purpose
Expose stable integration points for websites, partners, accountants, access vendors and automation tools.

Data model

- `ApiClient`: `id`, `name`, `scopes[]`, `siteIds[]`
- `ApiKey`: `id`, `clientId`, `status`, `lastUsedAt`
- `WebhookEndpoint`: `id`, `url`, `secret`, `subscriptions[]`, `status`
- `WebhookDelivery`: `id`, `endpointId`, `eventId`, `status`, `attempts`

API surface

- `POST /api/operator/v1/developer/api-keys`
- `POST /api/operator/v1/developer/webhooks`
- `GET /api/system/v1/events`
- `POST /api/system/v1/webhooks/{id}/replay`

Auth:

- Owner only by default
- Signed webhook deliveries

Sample request/response:

```
{
  "request": {
    "name": "accounting-sync",
    "scopes": ["invoices:read", "payments:read"],
    "siteIds": ["site_01"]
  },
  "response": {
    "clientId": "cli_01",
    "apiKeyPreview": "sk_live_****"
  }
}
```

UI flows and wireframes

- Owner: developer settings page with API keys, webhook logs, replay button and event catalogue.
- Operator: no default edit UI; optional read-only integration status.
- Tenant: no direct developer UI.

Error cases

- Invalid webhook signature
- Duplicate event replay causing side effects
- Rate limit exhausted
- Scope mismatch

Business rules

- API is versioned and backward-compatible within a major version.
- All write endpoints support idempotency.
- Webhooks are at-least-once delivered; consumers must be idempotent.
- Sensitive data is omitted unless scope explicitly permits.

Retention and GDPR

- Delivery logs retain only what is needed for debugging and audits, then roll off or redact payloads. [32]

Test cases

- Webhook retry and replay
- Key rotation
- Expired key access denial
- Scope-enforced object filtering

Monitoring and SLAs

- Webhook first-attempt latency
- Delivery success rate
- API error budget burn
- Rate-limit breach counts

Non-functional requirements

- Strong signatures
- Key rotation support
- Event schema versioning
- Request tracing IDs

Authentication and RBAC

Purpose
Secure every actor type and enforce site-scoped permissions.

Data model

- `User`: `id`, `type`, `email`, `mfaState`, `status`
- `Role`: `id`, `name`, `permissions[]`
- `PermissionAssignment`: `id`, `userId`, `roleId`, `siteIds[]`
- `Session`: `id`, `userId`, `device`, `expiresAt`
- `OrganisationMembership`: `id`, `organisationId`, `contactId`, `portalRole`

API surface

- `POST /api/operator/v1/users/invite`
- `POST /api/operator/v1/roles`
- `POST /api/operator/v1/users/{id}/assignments`
- `GET /api/operator/v1/auth/me`

- `POST /api/tenant/v1/auth/register`
- `POST /api/tenant/v1/auth/magic-link`

Auth:

- Bootstrapped owner
- OIDC for staff
- Tenant auth separate but same identity core

Sample request/response:

```
{
  "request": {
    "email": "ops@example.com",
    "displayName": "Marta",
    "role": "operator",
    "siteIds": ["site_01"]
  },
  "response": {
    "userId": "usr_01",
    "status": "invited"
  }
}
```

UI flows and wireframes

- Owner: user list with site assignment matrix, MFA state, invite/resend/deactivate actions.
- Operator: own profile and MFA setup page.
- Tenant: login/register page, organisation contact manager, saved session devices.

Error cases

- Invitation to existing user under wrong workspace
- Operator attempting privilege escalation
- Tenant contact removed while still sole organisation admin
- Reused magic link

Business rules

- Primary visible user types remain Owner, Operator and Tenant.
- Under the hood, permissions are granular and site-scoped.
- MFA is mandatory for staff users.
- Organisation accounts can have multiple contacts with different portal roles.
- Deactivating a user never removes their audit history.

Retention and GDPR

- User audit must remain even after deactivation for accountability, while personal profile data should still follow minimisation and deletion rules where permissible. [33]

Test cases

- Site-scoped operator sees only assigned site
- Owner invites and deactivates operator
- Tenant magic link flow
- Organisation account with two contacts and one payer

Monitoring and SLAs

- Failed login spikes
- MFA setup completion
- Permission-denied event trends
- Suspicious session activity

Non-functional requirements

- OIDC compliant
- Secure session rotation
- IP/device anomaly checks
- Auditability of permission changes

Notifications

Purpose
Send transactional e-mails, SMS and in-app messages for bookings, billing, access and operations.

Data model

- `NotificationTemplate`: `id`, `channel`, `locale`, `eventType`
- `NotificationPreference`: `id`, `userId`, `channel`, `enabled`
- `OutboundMessage`: `id`, `eventType`, `channel`, `status`, `providerRef`
- `NotificationEvent`: `id`, `type`, `subjectRef`, `createdAt`

API surface

- `POST /api/operator/v1/notifications/test`
- `PATCH /api/operator/v1/notification-templates/{id}`
- `PATCH /api/tenant/v1/notification-preferences`
- `GET /api/operator/v1/outbound-messages`

Auth:

- Owner/Operator for templates and operational message review
- Tenant for own preferences

Sample request/response:

```
{
  "request": {
    "eventType": "invoice_overdue",
    "channel": "email",
```

```
    "recipientUserId": "cust_01"
  },
  "response": {
    "messageId": "msg_01",
    "status": "queued"
  }
}
```

UI flows and wireframes

- Owner: template editor with locale tabs and preview payload.
- Operator: resend/override actions on invoice or agreement timeline.
- Tenant: notification preferences and in-app inbox.

Error cases

- Hard bounce
- SMS delivery failure
- Invalid locale fallback
- Duplicate send after retry

Business rules

- Transactional notices are separate from marketing consent.
- Reminder schedules are policy-driven from billing module.
- Access and lockout notices must be clearly distinguished from promotional messages.

Retention and GDPR

- Message bodies containing personal data should have bounded retention and redaction where practical.
- Marketing consent and unsubscribe state must be auditable. [34]

Test cases

- Invoice reminder schedule
- Access restored notice after payment
- German/English template locale fallback
- Bounce suppression

Monitoring and SLAs

- Delivery success by channel
- Bounce and complaint rates
- Queue latency

Non-functional requirements

- Provider abstraction
- Rate limiting
- Template versioning
- Delivery traceability

Backups, audit and logging

Purpose
Protect recoverability, provide tamper-resistant history and support compliance and incident investigation.

Data model

- `AuditEvent`: `id`, `actorId`, `action`, `subjectType`, `subjectId`, `changes`, `createdAt`
- `BackupSet`: `id`, `kind`, `startedAt`, `completedAt`, `checksum`
- `RestoreJob`: `id`, `requestedBy`, `scope`, `status`
- `SecurityEvent`: `id`, `type`, `severity`, `details`
- `LegalHold`: `id`, `subjectRef`, `reason`, `active`

API surface

- `GET /api/operator/v1/audit`
- `GET /api/system/v1/backups/status`
- `POST /api/system/v1/restore-jobs`
- `GET /api/system/v1/security-events`

Auth:

- Owner only for restore initiation
- Operator read-only audit where allowed
- Security endpoints restricted

Sample request/response:

```
{
  "request": {
    "scope": "site_01",
    "restorePoint": "2026-05-09T00:00:00Z"
  },
  "response": {
    "restoreJobId": "rst_01",
    "status": "pending_approval"
  }
}
```

UI flows and wireframes

- Owner: audit explorer with actor, entity and site filters; backup health card; restore request workflow.
- Operator: read-only subject history from customer/unit/invoice pages.
- Tenant: account activity list for own logins and profile changes.

Error cases

- Backup checksum mismatch
- Restore attempted without dual approval
- Missing audit for privileged action
- Logging pipeline outage

Business rules

- Every privileged write emits an audit event.
- Backup restore requires break-glass approval.
- Legal holds stop deletion/anonymisation jobs.
- Security events are separated from general app logs.

Retention and GDPR

- Audit and security logs must balance accountability with minimisation; payloads should avoid unnecessary PII where possible.
- Deletion/return obligations under controller-processor contracts must be supportable operationally. [32]

Test cases

- Backup creation and restore drill
- Audit completeness for role changes
- Legal hold prevents anonymisation
- Security alert on repeated failed logins

Monitoring and SLAs

- Backup success rate
- Restore drill frequency
- Audit event pipeline lag
- Security alert MTTR

Non-functional requirements

- Immutable audit sink
- Encrypted backups
- Periodic restore testing
- Time-synchronised event ordering

Deployment and infrastructure

Purpose
Run the product reliably in production with safe releases, EU-oriented data handling and predictable operations.

Data model

- `Environment`: `name`, `region`, `version`
- `Deployment`: `id`, `service`, `version`, `status`

- `SecretRef`: `id`, `service`, `rotatedAt`
- `WorkerQueue`: `name`, `lag`, `throughput`
- `FeatureFlag`: `id`, `name`, `scope`, `state`

API surface

- `GET /healthz`
- `GET /readyz`
- `GET /metrics`
- internal release/flag endpoints only

Auth:

- Internal ops only
- No tenant exposure beyond status communication

Sample request/response:

```
{
  "response": {
    "status": "ok",
    "version": "0.1.0",
    "db": "ok",
    "queue": "ok"
  }
}
```

UI flows and wireframes

- Owner: optional system status widget and incident banner.
- Operator: service-status banner for degraded workflows.
- Tenant: public status/maintenance banner only when relevant.

Error cases

- Region outage
- Queue backlog causing delayed notifications or billing
- Failed migration
- Secret rotation drift

Business rules

- Primary production data stays in EU-region infrastructure where possible.
- Blue/green or canary deployments for user-facing services.
- No schema-breaking deploy without backward-compatible migration path.
- Feature flags gate rollout of risky changes per site or workspace.

Retention and GDPR

- Primary data placement and backups should remain inside EU-capable infrastructure unless a documented transfer basis exists.

- Sub-processor inventory and DPAs must be maintained. [35]

Test cases

- Blue/green deployment
- Rollback after migration issue
- Regional failover drill
- Queue saturation alert

Monitoring and SLAs

- Errors, latency, throughput, saturation for each service
- Queue lag and dead-letter metrics
- Release health score by version
- SLO and error-budget reporting

Non-functional requirements

- Infrastructure as code
- Secret manager integration
- Automated backups and restore drills
- Centralised observability
- EU-region object storage and backups where possible

Preferred technical choices

Application architecture

- Front end: React [4] with a server-capable framework
- Back end: Node.js [5] + TypeScript [6]
- Pattern: modular monolith first, with explicit module boundaries and domain events

Core infrastructure

- Database: PostgreSQL [20]
- Queue: durable job queue with repeatable jobs and dead-letter support
- Object storage: S3-compatible, EU region
- Search: Postgres full-text first; dedicated search service only if scale justifies it
- Identity: OIDC-based IdP, staff MFA mandatory, optional self-hosted Keycloak [36] path if data/control requirements demand it
- Observability: traces, metrics and structured logs from day one

Payment providers

- Primary abstraction should support cards, wallets, invoice/manual methods and SEPA.
- Good initial candidates are and .
- If the operator later needs a bank-debit-first flow, evaluate carefully, noting its documented SEPA Core focus. [37]

Accounting integration pattern

- MVP: file-based DATEV-compatible export
- v1: richer export mapping and accountant-friendly bundles

- v2: optional direct DATEV services integration via where the operator and accountant want it. [38]

Access integration pattern

- Adapter-per-vendor model
- Desired event contract: `credential.issued`, `credential.revoked`, `access.denied`, `access.used`, `vendor.offline`
- Reconciler jobs compare internal intended state with vendor actual state

Decision log template

```
## Decision log

| Date | Version | Topic | Options considered | Decision | Why | Consequences
| Owner |
|---|---|---|---|---|---|---|---|
| 2026-05-09 | 0.1.0 | Primary billing model | Fixed-date monthly vs
anniversary | Fixed-date monthly first | Simpler operations and accounting |
Anniversary mode deferred | Product |
```

Roadmap and implementation order

The MVP should focus on the smallest coherent operating system that can replace spreadsheets, e-mail coordination and gate-code workarounds for a two-site operator. That means multi-site inventory, public storefront, live availability, reservations, agreement generation and signing, tenant portal, recurring billing, payment and mandate handling, overdue workflows, at least one access-control adapter, inspection/task workflows, core reporting, audit trail and file-based German accounting exports. This set is enough to create a commercially credible and legally aligned first product. It is also the minimum set implied by what leading storage platforms already treat as standard. [39]

The first major release after MVP should deepen the operating model rather than broaden it blindly. Priorities should be dynamic pricing and scheduled rent increases, organisation-account workflows with multiple contacts and authorisations, stronger document/version controls, full public API and signed webhooks, better reconciliation tooling, multiple access vendors, advanced inspection/incident workflows, and more robust multilingual communications. Those are the capabilities that most directly improve revenue and operator confidence without changing the core product thesis. [40]

The second major release should add optional depth where the operator's real business proves it is needed: deeper DATEV service integration, stronger SEPA B2B support if demanded, richer portfolio analytics, embedded support tooling, additional upsells, and potentially adjacent logistics models such as vehicle spaces or pickup-and-return storage. By that point the architecture should still remain modular, but still monolithic unless operational scale has genuinely justified a split. The main risk to avoid is premature diversification into "anything rentable"; the moat is the storage operating system, not rental generalism. [41]

---

[1] [15] [16] [18] [39] https://stora.co/de/
https://stora.co/de/

2 8 https://www.selfstorage-verband.de/self-storage-verband
https://www.selfstorage-verband.de/self-storage-verband

3 4 11 23 28 https://www.bundesfinanzministerium.de/Content/DE/FAQ/e-rechnung.html
https://www.bundesfinanzministerium.de/Content/DE/FAQ/e-rechnung.html

5 27 32 35 https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/what-should-be-included-controller_en
https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/what-should-be-included-controller_en

6 21 36 37 https://docs.stripe.com/payments/sepa-debit?locale=en-GB
https://docs.stripe.com/payments/sepa-debit?locale=en-GB

7 24 25 26 30 34 https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/what-are-basic-processing_en
https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/what-are-basic-processing_en

9 https://lageris.de/
https://lageris.de/

10 41 https://www.storagebook.de/
https://www.storagebook.de/

12 https://www.bundesbank.de/de/aufgaben/unbarer-zahlungsverkehr/serviceangebot/sepa/inhalte/inhalte-642840?index=3
https://www.bundesbank.de/de/aufgaben/unbarer-zahlungsverkehr/serviceangebot/sepa/inhalte/inhalte-642840?index=3

13 https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/do-data-processors-also-have_en
https://www.edpb.europa.eu/sme-data-protection-guide/faq-frequently-asked-questions/answer/do-data-processors-also-have_en

14 20 31 https://eur-lex.europa.eu/legal-content/EN-ES/TXT/?uri=CELEX%3A32014R0910
https://eur-lex.europa.eu/legal-content/EN-ES/TXT/?uri=CELEX%3A32014R0910

17 https://help.storedge.com/hc/en-us/articles/208591876-User-Management-and-Access
https://help.storedge.com/hc/en-us/articles/208591876-User-Management-and-Access

19 https://stora.co/de/features/self-storage-management-software
https://stora.co/de/features/self-storage-management-software

22 29 38 https://www.datev.de/web/de/datev-shop/91000-buchungsdatenservice/
https://www.datev.de/web/de/datev-shop/91000-buchungsdatenservice/

33 https://www.edpb.europa.eu/sme-data-protection-guide/data-controller-data-processor_en
https://www.edpb.europa.eu/sme-data-protection-guide/data-controller-data-processor_en

40 https://www.stora.co/
https://www.stora.co/